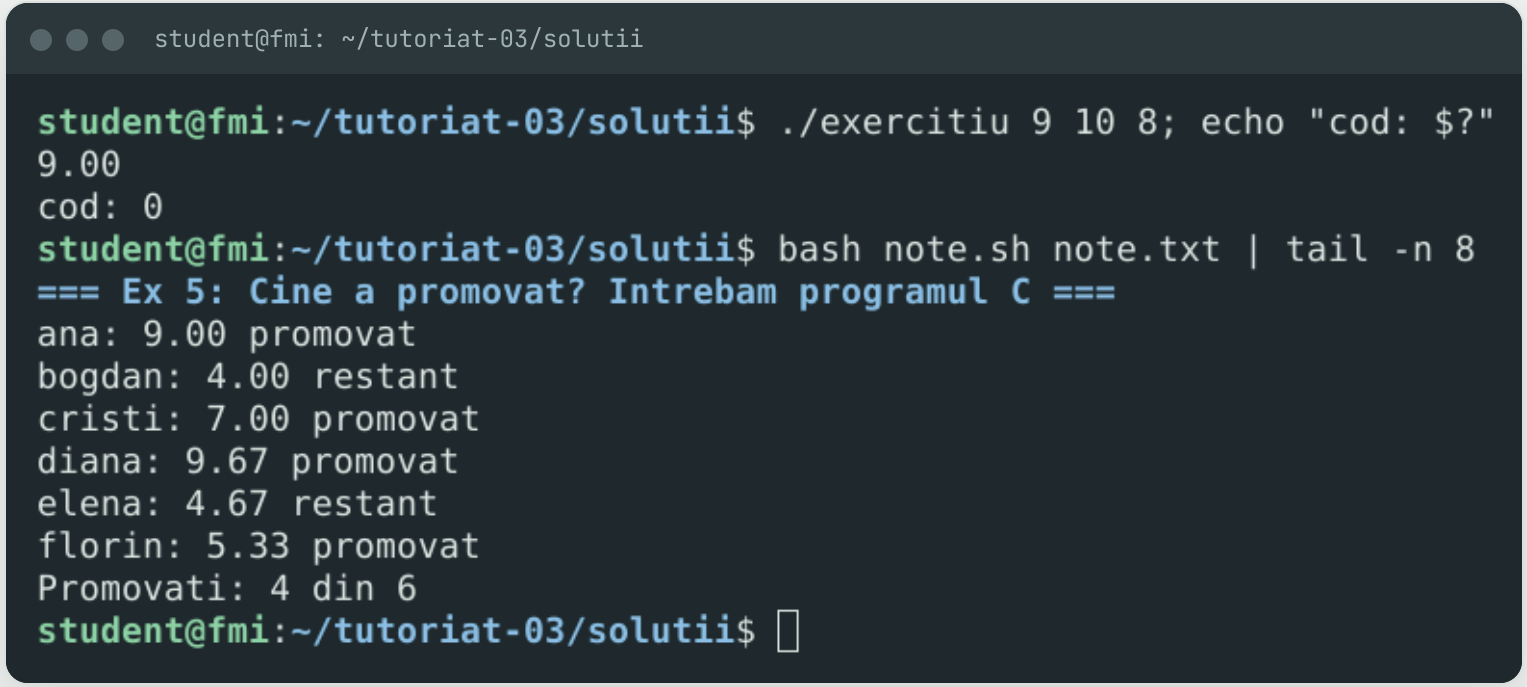
Primele scripturi în Bash

Scriem comenzile într-un fișier ca să le rulăm dintr-o singură mișcare: cu variabile, decizii și bucle. La final, scriptul nostru întreabă un program C cine a promovat.



Captură reală. Ăsta e rezultatul ultimului exercițiu de azi.

PLANUL DE AZI

Cinci întrebări, apoi trecem la terminal

1

Ce e un script?

Un fișier cu comenzi. `bash f.sh` sau `./f.sh`.

2

Cum țin minte o valoare?

Variabile și argumente: `$1`, `$#`.

3

Cum calculez?

`$( )` prinde ce afișează o comandă, `$(( ))` face calcule.

4

Cum iau o decizie?

`if [ ... ]` și codul de ieșire.

5

Cum repet ceva?

Buclele `for` și `while read`.

Cuvinte pe care le vom folosi des. Apasă pe unul:

script · shebang · variabilă · argument · $( ) · $(( )) · cod de ieșire · test / [ ] · buclă

script Un fișier text cu comenzi pe care shell-ul le rulează una după alta, de sus în jos.

Cum împărțim cele 90 de minute

15′ idei	60′ lucrăm în terminal	10′ Kahoot	5′

1 · CE E UN SCRIPT

Un fișier text cu comenzi, rulate de sus în jos

Exact ce ai scrie de mână în terminal, doar că salvat. Apasă pe comenzi în ordine.

salut.sh `-rwxr-xr-x`

```
1 #!/bin/bash
2 # salut.sh: primul script
3 echo "Salut, $USER!"
4 echo "Esti in $(pwd)"
```

`#!/bin/bash` se numește shebang: îi spune sistemului cu ce program rulează fișierul. Liniile cu # sunt comentarii.

`bash salut.sh` `./salut.sh` `chmod +x salut.sh`

`De la capăt`

```
$ bash salut.sh
Salut, student!
Esti in /home/student/tutoriat-03
$ chmod +x salut.sh
$ ./salut.sh
Salut, student!
Esti in /home/student/tutoriat-03
```

Sistemul citește prima linie, `#!/bin/bash`, și pornește `bash` cu fișierul tău.

CA ȘI CUM scrii o rețetă pe un bilet în loc s-o spui de fiecare dată. `bash salut.sh` = „citește biletul cu voce tare”. `./salut.sh` = biletul se citește singur, dar doar dacă are voie (`chmod +x`).

nume=ana: fără spații în jurul lui =

O variabilă e o etichetă pusă pe un text. O citești cu $ în față. Apasă comenzile pe rând.

```
nume=ana
```

```
echo "Salut, $nume"
```

```
echo 'Salut, $nume'
```

```
nume = ana
```

```
nume=Ana Maria
```

```
nume="Ana Maria"
```

De la capăt

Ce variabile ține minte shell-ul acum:

nume	ana

```
$ nume=ana
$ echo "Salut, $nume"
Salut, ana
$ echo 'Salut, $nume'
Salut, $nume
$ nume = ana
bash: nume: command not found
```

Cu spații, shell-ul crede că `nume` e o comandă cu argumentele `=` și `ana`.

> Toate variabilele sunt **text**. `x=5` pune în `x` caracterul „5”. Pentru calcule avem `$(( ))`, două slide-uri mai încolo.

2 · ARGUMENTELE SCRIPTULUI

Ce scrii după numele scriptului ajunge în $1, $2...

Exact ca argv în C. Alege o linie de comandă:

`bash arg.sh note.txt extra` | `bash arg.sh` | `bash arg.sh "fisier cu spatii.txt"`

`bash arg.sh fisier cu spatii.txt` | `bash arg.sh a b c d`

VARIABILA	VALOAREA	ÎN C
$0	arg.sh	argv[0]
$1	note.txt	argv[1]
$2	extra	argv[2]
$#	2	argc - 1
$@	note.txt extra	argv[1..]

arg.sh

```
1 #!/bin/bash
2 echo "\$0 = $0"
3 echo "\$1 = $1"
4 echo "\$2 = $2"
5 echo "\$# = $#"
6 echo "\$@ = $@"
```

Două argumente: $# este 2.

3 · CALCULE

O paranteză prinde text, două paranteze fac calcule

$(comanda)

Rulează comanda și pune în loc ce a afișat ea.

```
n=$(wc -l < note.txt); echo "Sunt $n studenti"
```

```
echo "Fisiere: $(ls *.txt)"
```

```
media=$(./exercitiu 9 10 8)
```

$((expresie))

Calcul cu numere **întregi**. Înăuntru nu-i nevoie de $.

```
echo $(( 7 * 3 ))
```

```
echo $(( 29 / 3 ))
```

```
echo $(( 29 % 3 ))
```

```
i=5; i=$(( i + 1 )); echo $i
```

```
echo 7 + 3
```

```
$ n=$(wc -l < note.txt); echo "Sunt $n studenti"
Sunt 6 studenti
```

`wc -l` a afișat 6; `$( )` a pus acel text în `n`.

`if` se uită doar la codul de ieșire

`[` e o comandă (`test`): răspunde cu `0` = adevărat sau `1` = fals. Alege un test (în folder există `note.txt`, `$#` e 1, `nume=ana`).

```
[ -f note.txt ]
```

```
[ -f lipsa.txt ]
```

```
[ ! -f lipsa.txt ]
```

```
[ $# -ne 1 ]
```

```
[ "$nume" = ana ]
```

```
[ 10 -gt 9 ]
```

```
[ 10 > 9 ]
```

```
grep -q ana note.txt
```

```
if [ -f note.txt ]; then
    echo "da"
else
    echo "nu"
fi
```

0 Fișierul există, deci testul răspunde 0 (adevărat).

`-f f`	fișierul există
`-d d`	directorul există
`-x f`	se poate executa
`-z "$s"`	textul e gol
`a = b`	texte egale
`-eq -ne`	numere = ≠
`-lt -gt`	numere < >
`-le -ge`	numere ≤ ≥
`!`	negație

> **Spații obligatorii:** `[ $# -ne 1 ]`. Fără ele, `[$#` devine un singur cuvânt și primești `[1: command not found`.

5 · BUCLE

`for` ia pe rând fiecare cuvânt dintr-o listă

`for f in *.c *.sh *.txt` `for i in 1 2 3 4 5` `for nume in ana bogdan cristi`

```
for f in *.c *.sh *.txt; do
    echo "$f: $(wc -l < "$f") linii"
done
```

Lista, după ce shell-ul a înlocuit măștile *.c *.sh *.txt:

exercitiu.c note.sh **note.txt**

f	note.txt

Pasul următor De la capăt

```
exercitiu.c: 84 linii
note.sh: 64 linii
note.txt: 6 linii
```

5 · BUCLE

`while read` citește un fișier linie cu linie

```
while read -r nume n1 n2 n3; do
    echo "$nume: $(( (n1 + n2 + n3) / 3 ))"
done < note.txt
```

note.txt

```
ana 9 10 8
bogdan 4 5 3
cristi 7 6 8
diana 10 9 10
elena 5 4 5
florin 6 5 5
```

Linia următoare

De la capăt

`read` împarte linia la spații:

nume	n1	n2	n3
diana	10	9	10

`$(( (10 + 9 + 10) / 3 )) = $(( 29 / 3 )) = 9`

Media reală e 9.67, dar Bash taie zecimalele. De aceea, în exercițiul 5, media o calculează programul C.

```
ana: 9
bogdan: 4
cristi: 7
diana: 9
```

EXERCIȚIUL DE AZI

Un program C pe care îl poate folosi un script

`./exercitiu <nota1> [nota2 ...]` calculează media notelor primite ca argumente.

1. Notele vin în `argv`.

   `argv[1]` e prima notă, cum în script `$1` e primul argument.

2. Pe `stdout` doar media, cu 2 zecimale.

   Ca scriptul s-o prindă cu `media=$(./exercitiu 9 10 8)`.

3. Erorile pe `stderr`.

   Fără note sau cu `abc`: mesaj de eroare, nu media.

4. Cod de ieșire: 0 promovat, 1 restant, 2 eroare.

   Scriptul îl citește cu `$?`.

Aceleași idei, în C și în Bash:

`argc - 1`	↔	`$#`
`argv[1]`	↔	`$1`
`return 1;`	↔	`$?`

EXERCIȚIUL DE AZI

Soluția, rulată din terminal și din script

```
./exercitiu 9 10 8; echo $?
```

```
./exercitiu 4 5 3; echo $?
```

```
./exercitiu 10 9 10
```

```
./exercitiu 7 abc; echo $?
```

```
./exercitiu; echo $?
```

```
bash note.sh note.txt | tail -n 7
```

```
$ ./exercitiu 9 10 8; echo $?
9.00
0
```

stdout stderr promovat: cod 0

GREȘELI FRECVENTE

Capcanele clasice din primele scripturi

Apasă pe una: vezi de ce se întâmplă și cum o eviți.

`nume = ana → nume: command not found` ⌄

DE CE

Cu spații în jurul lui =, shell-ul crede că nume e o comandă cu argumentele = și ana.

REZOLVARE

`nume=ana`

`[$# -ne 1] → [1: command not found` ›

`[ 10 > 9 ] → apare un fișier numit 9` ›

`echo $(( 29 / 3 )) → 9` ›

`[ -f $1 ] → adevărat chiar dacă n-ai dat niciun argument` ›

`echo "media: $media"; if [ $? -eq 0 ] → mereu 0` ›

ÎNCĂLZIRE ÎNAINTE DE KAHOOT

Trei întrebări rapide

Rulezi bash s.sh a b c. Cât este $#?

- 4
- 3 ✓
- 0

Corect. $# numără doar argumentele, fără numele scriptului ($0).

Ce afișează echo $((7 / 2))?

- 3.5
- 3
- 4

Care atribuire este corectă?

- x = 5
- $x=5
- x=5

ACUM E RÂNDUL TĂU

Deschide terminalul și urmează pașii

Întâi exercițiile 1–4 din `note.sh` (doar Bash), apoi `TODO`-urile din scheletul `exercitiu.c`, apoi exercițiul 5.

- [ ] `cd tutoriat-03/schelet` intră în folder
- [ ] `bash note.sh note.txt` exercițiile 1–4
- [ ] `nano exercitiu.c` rezolvă TODO 1–5
- [ ] `make && make test` compilează și testează
- [ ] `make script` exercițiul 5: „Promovati: 4 din 6”

Ce exersezi

Citești argumentele scriptului, iei decizii cu `if`, parcurgi un fișier cu `while read` și folosești un program C prin ce afișează (`$( )`) și prin codul lui de ieșire (`$?`).

Te-ai blocat?

Fiecare `todo` are un indiciu deasupra. Soluțiile sunt în `tutoriat-03/solutii/`. Încearcă întâi singur.